DEFENSIVE SECURITY WORKBENCH

CyberShield AI

AI-Assisted Security Log Triage and Analysis

Understand the signal. Inspect the evidence. Keep a human in control.

LOG INGESTION | EXPLAINABLE SCORING | LOCAL ANOMALY MODEL | CSV / JSON EXPORT

PROJECT REPORT

Student: Devulapalli Akhil
Institution: GITAM
Department: CSE Cyber Security
Submission date: 22 October 2026

EDUCATIONAL PROTOTYPE · HUMAN REVIEW REQUIRED

01 Executive summary

CyberShield AI is a local-first workbench that helps a reviewer turn noisy authentication and system events into a readable triage queue. It normalizes common log formats, correlates repeated failed logins, gives each event a bounded risk score, and presents the evidence behind that score. The reviewer can investigate details and export results for further analysis.

DESIGN PRINCIPLE Every score must be explainable, every recommendation must be defensive, and every response decision stays with a human reviewer.

Project objectives

• **Ingest:** parse bounded JSON/JSONL, CSV, and a practical subset of OpenSSH authentication logs.

• **Detect:** identify authentication bursts, username spraying, success after a burst, privileged activity, sensitive paths, and selected suspicious strings.

• **Explain:** show detection evidence, risk contribution, severity, and conservative next steps.

• **Explore:** filter events and visualise event counts and risk distribution in an approachable multi-colour interface.

• **Export:** create event-level CSV / JSON and aggregate summary JSON for reports or offline analysis.

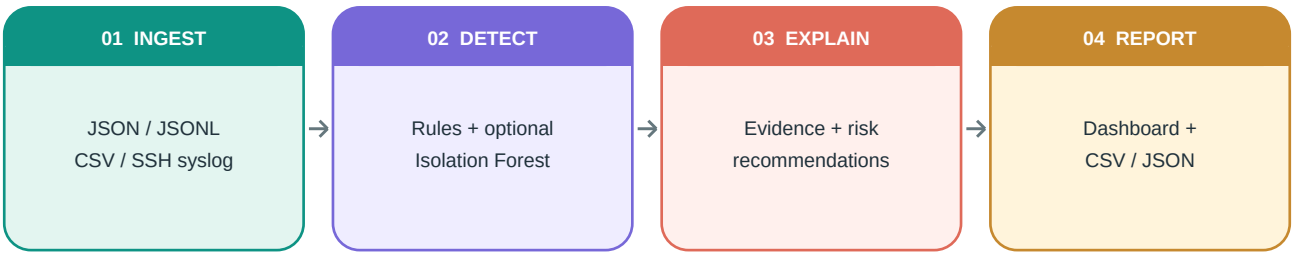
• **Respect data:** perform local analysis with no external API calls or automated response actions.

Project snapshot

Area	Implementation	Boundary
Ingestion	JSON, JSONL, CSV, SSH syslog subset	Unknown formats are reported; no guessed semantics.
Detection	Explainable rule scoring + optional Isolation Forest	No labelled threat-training corpus or accuracy claim.
Recommendations	Deterministic evidence-linked defensive checklists	No third-party generative AI calls.
Reporting	Dashboard, CSV, JSON exports	Exports may contain sensitive source log fields.

02 Architecture and data flow

The UI is kept separate from the core pipeline. This makes parsing, scoring, recommendations, and exports testable without launching the web interface.



Local process · no external API calls · no automatic remediation

Module responsibilities

Module	Responsibility
app.py	Streamlit navigation, uploads, dashboard metrics/charts, filters, event details, and download controls.
cybershield.models	Typed event, parse-issue, and risk-signal structures.
cybershield.ingest	Input-size checks, field normalization, IP validation, format parsers, and visible parsing issues.
cybershield.detection	Batch correlation, evidence strings, risk weights, severity bands, and aggregate counts.
cybershield.anomaly	Optional Isolation Forest fitted to the loaded batch; outlier score and flag only.
cybershield.recommendations	Rule-linked defensive follow-up prompts designed for human verification.
cybershield.reporting	CSV / JSON serialization with structured columns and aggregate output.

Supported inputs

JSON / JSON array: a single event object or a list of objects. **JSON Lines:** one event per non-empty line. **CSV:** header-based records. **OpenSSH syslog subset:** accepted/failed SSH authentication and simple sudo records. Common aliases such as *user* / *username*, *time* / *timestamp*, and *src_ip* / *source_ip* are normalized.

Input safety defaults: 5 MiB maximum file size, 20,000 records, and bounded message fields. Unknown raw syslog lines create parser notes rather than being silently interpreted.

03 Detection and risk scoring

Rule points are added when evidence is observed, then capped at 100. Multiple indicators may overlap on a single event. The intent is to prioritize review, not to assign a definitive maliciousness label.

Signal	Default points	What the reviewer should verify
Failed authentication	+10	Whether the attempt is expected and whether an account is affected repeatedly.
Repeated failure burst	+20 to +30	Five or more failed logins from one source in a configurable 15-minute window.
Username spraying	+15	Multiple account names attempted from the same source.
Success after failure burst	+18	Whether the subsequent session belongs to the account owner.
Administrative account	+8	Expected use, approved automation, and account ownership.
Privilege-related event	+28	Command, change ticket, authorisation, and least-privilege policy.
Malware / credential alert type	+35	The originating alert and corroborating endpoint records.
Sensitive file / suspicious command	+20 to +22	Context around credential stores, sensitive paths, or command patterns.
Unusual UTC hours	+7	User schedule, change windows, and known jobs.
Dataset-relative outlier	+8	Whether the event differs meaningfully from its current batch.

Severity bands

Low · 0–24	Guarded · 25–49	High · 50–74	Critical · 75–100
Routine context; retain for correlation.	Some indicators; review with context.	Prioritise corroboration and owner validation.	Prompt urgent analyst validation under local policy.

Optional anomaly model

Isolation Forest is applied to features derived from the current batch, including result, account category, event type, message length, source frequency, repeated failures, selected text indicators, and hour. It requires at least 20 records and returns a dataset-relative outlier cue. It is not trained on labelled attacks, and a flagged record is not proof of compromise. No precision, recall, accuracy, or latency claim is made.

04 Human-centred interface and safeguards

The dashboard uses a calm paper-like background and a distinct palette: teal for normal navigation and local analysis, violet for model-related context, amber for caution, and coral/red for higher-risk findings. Colour is paired with text labels and numeric scores so the interface is not colour-only.

Workspace	Primary purpose
Command centre	Event totals, severity distribution, time trend, and a short high-priority list.
Event explorer	Filter on triage level, outcome, event type, and text fields; inspect the message, evidence, and defensive actions.
Detection lab	Understand signal frequency, default scoring rules, and the anomaly model boundary.
Reports & exports	Download event-level CSV/JSON and aggregate summary JSON.
About & safety	Architecture, limitations, authorisation requirements, and safe demonstration practices.

Privacy and responsible use

• The app makes no outbound API calls, uses no third-party AI endpoint, and does not save uploaded source logs to disk.

• Uploads are processed in the local Streamlit process. Downloaded exports can contain usernames, addresses, host names, and message text; review before sharing.

• The app does not block traffic, disable accounts, execute log content, or perform automatic remediation.

• Use only logs you are authorised to inspect. Preserve context and seek corroboration from identity, endpoint, network, and change-management data.

• The bundled source examples use documentation IP ranges and are synthetic, not operational telemetry.

LIMITATION TO SAY OUT LOUD IN THE DEMO Rules can produce false positives and false negatives. Anomaly scores are batch-relative. Absence of an alert does not establish safety, and the application is not a production SIEM.

Deployment hardening needed before real use

A real deployment needs authentication, role-based access, transport security, audit trails, retention policy, resource monitoring, dependency scanning, approved source schemas, and a documented incident-response process. Do not expose the development server directly to the public internet.

05 Setup, verification, and demonstration

Install the dependencies in a dedicated virtual environment from the project root.

Windows / VS Code terminal

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

Verification record (build environment)

Check	Result	Honest interpretation
Python compile check	Passed	app.py, package modules, and tests compile with compileall.
Pytest unit tests	15 passed	Parser, detection, scoring, exports, anomaly return shape, and a structured-outcome edge case were exercised.
Synthetic JSONL fixture	59 records; 0 parse issues	Parsed and analysed with the programmatic pipeline. Counts are fixture-specific, not performance statistics.
Synthetic OpenSSH fixture	10 records; 0 parse issues	The bundled syslog subset parsed successfully.
Streamlit browser startup	Not verified here	Streamlit was unavailable in the build container and package installation could not reach the package index. Launch it after installing requirements.

Practical demonstration checklist

- ■ Create the virtual environment and install requirements.
- ■ Run python -m pytest -q and keep the real terminal output.
- ■ Launch the dashboard and show the synthetic-data label.
- ■ Explain the repeated failed-login burst from 203.0.113.77.
- ■ Open the successful login after the burst and explain the correlation evidence.
- ■ Toggle the anomaly model and state why an outlier is not proof of attack.
- ■ Upload data/sample_auth.log and check parser notes.
- ■ Export event CSV and JSON and open the files.
- ■ Describe limitations and the human approval requirement.

Limitations and next research steps

The parser covers a subset of OpenSSH and sudo records. Syslog usually omits year and timezone; this implementation assumes UTC and assigns the current UTC year. Scoring defaults require tuning. The model has no labelled reference corpus. There is no durable log store, multi-user access control, real-time tailing, alert delivery, or production workload validation. Future work should use authorised labelled data, measure precision/recall and analyst alert load, extend schema coverage, and test privacy/access controls.